Trade-offs & Production Readiness Analysis

Mal Credit Decision Data Platform — Part 2

Priyanka | Credit & Lending Data Engineering Assessment

This analysis is grounded in the Part 1 implementation: a medallion pipeline (bronze → silver → gold) ingesting AECB XML, fraud API scores, AML webhooks, and internal PostgreSQL profiles into a unified credit decision record. All trade-off references below map directly to specific design choices in that codebase.

1 Conflict Resolution Trade-offs

How I handled customer key conflicts

The core challenge is that each of the four sources uses a different identifier for the same person. I built a four-tier cascade in *python/transformation/identity_resolver.py*: internal UUID first, then Emirates ID, then phone + email together, and finally fuzzy name + date of birth as a last resort. The order reflects confidence — our own UUID is the most reliable because we issued it; the government-issued Emirates ID is next; phone and email together are decent but not bulletproof; name matching is the weakest and always gets flagged for manual review.

Edge cases and failure modes I thought through

- **Same Emirates ID, different names:** Can happen with data entry errors or name changes after marriage. The conflict checker in *_check_conflicts()* catches this and logs it to *silver.identity_resolution_exceptions* rather than silently overwriting.
- **Shared phone numbers:** Family plans in UAE are common — a father and son might share a number. This is why I require both phone AND email to match at Tier 3, not just one.
- **Arabic name transliteration:** 'Mohammed', 'Mohammad', 'Mohamed' all refer to the same person but spell differently depending on the document. I landed on a 0.85 pg_trgm similarity threshold after testing on ~200 sample name pairs — low enough to catch transliteration variants, high enough to avoid false matches between different people.
- **New customer with no match at any tier:** The resolver creates a new customer_key. The risk here is duplicate records if the same person applies twice via different channels before we link them. This is tracked via the exception log and reviewed weekly.
- **Webhook arrives before profile CDC:** AML can screen a customer before their profile lands in bronze. The resolution step handles this — AML records with no match go to exceptions and are re-evaluated on the next pipeline run once the profile arrives.

What I would do differently at 10x scale (100K+ decisions/day)

At current volume, running similarity() inside PostgreSQL on every batch is manageable. At 10x, that becomes a bottleneck. The changes I would make:

- Move name matching to a dedicated entity resolution service (e.g. Zingg or Splink) running as a separate microservice — these are purpose-built for fuzzy matching at scale and significantly faster than SQL similarity functions.
- Add a Redis cache layer for customer_key lookups — most decisions are from returning customers whose UUID or Emirates ID is already in customer_master, so a cache hit avoids a DB round-trip entirely.

- Introduce a probabilistic matching score model trained on confirmed matches, replacing the fixed 0.85 threshold with a model-derived confidence score.
- Partition silver.customer_master by nationality or Emirates ID prefix to parallelise resolution queries.

2 Data Quality Strategy

Why I split rules into MUST_PASS vs HIGH vs warnings

Not every data quality problem should stop the pipeline — that would be overly conservative and block legitimate decisions. The distinction I drew is between rules that protect technical correctness and rules that protect business risk.

Rule	Severity	Why this level
DQ-AECB-01: Emirates ID format	MUST_PASS	A malformed ID breaks identity resolution entirely — no decisions possible
DQ-AECB-02: Credit score in 300–900	MUST_PASS	Score outside this range crashes the scoring model mathematically
DQ-FRAUD-01: Fraud score in [0,1]	MUST_PASS	Any value outside this range is a provider API error, not valid data
DQ-AML-01: Risk level not null	MUST_PASS	A null AML result means we have no idea if the customer is sanctioned
DQ-AECB-03: Report within 90 days	HIGH	A 91-day-old report is still usable — just flagged for underwriter review
DQ-FRAUD-02: Scored in last 24h	HIGH	Slightly stale score is acceptable; very stale (>48h) would be MUST_PASS
DQ-CUST-01: Income > 0	HIGH	Some customers legitimately have zero declared income (students, dependants)

Balancing decision speed vs data completeness

The design allows decisions to proceed even when non-critical sources are degraded. Each gold decision snapshot carries a *dq_score* (0–1) that reflects how complete the inputs were. A decision made with a stale AECB report gets a lower dq_score and lands in a REFERRED state for underwriter review, rather than being auto-approved or auto-declined. This means the pipeline keeps running during partial outages without compromising credit safety.

What happens when AECB data is delayed

AECB delivers via SFTP once daily. If the file doesn't arrive by the pipeline window (02:00 UAE), the Airflow DAG will complete but any applications relying on AECB data will have their decision pushed to REFERRED with a dq_flag of 'AECB_DATA_MISSING'. The downstream application flow routes these to a manual underwriting queue. This was a deliberate call — auto-declining because of a feed delay would unfairly penalise customers and create regulatory exposure.

> *Key principle: the pipeline distinguishes between 'we have bad data' (halt) and 'we have incomplete data' (continue with lower confidence and route to human review).*

3 Compliance & Auditability

How the design supports UAE Central Bank audits

The core CBUAE requirement is that every credit decision must be fully reconstructable — you need to be able to show exactly what data was used, as it was at the time. Three design choices in Part 1 make this possible:

- **Append-only bronze tables** — no UPDATE or DELETE is ever run on bronze. Every raw payload that ever arrived from AECB, fraud, or AML is permanently preserved with a timestamp and checksum.
- **Immutable gold snapshots** — *gold.credit_decision_inputs* stores a frozen copy of every input field at decision time. Even if the customer's credit score changes tomorrow, the record shows what the score was when the decision was made.
- **Full lineage chain** — every gold row has foreign keys back to specific silver rows (*silver_aecb_id, silver_fraud_id* etc.), and every silver row has a *bronze_id* pointing to the original raw payload. An auditor can trace any decision back to the raw XML byte.

The audit query is straightforward: given a *decision_id*, join through gold → silver → bronze for each source. The raw AECB XML, the fraud API JSON, the AML webhook payload, and the customer profile snapshot are all retrievable. Files are also archived on S3 (KMS encrypted, versioning on) for the 7-year retention period CBUAE requires for credit records.

Handling GDPR-style data deletion requests

This is a genuine tension in the design. A customer has the right to request erasure of their personal data, but CBUAE requires credit records to be retained for 7 years. My approach resolves this through pseudonymisation rather than deletion:

- **What gets erased:** Name, DOB, Emirates ID, phone, email — all PII fields — are replaced with an irreversible hash in silver.customer_master and silver.customer_profiles.
- **What gets retained:** The credit signals (scores, delinquency history, DTI ratios) and the decision outcome. These are not personal data in the regulatory sense — they are financial records the bank is legally required to keep.
- **Bronze raw payloads:** These contain PII and are legally required to be retained. The approach is to flag the S3 object with a deletion marker so it is excluded from any future processing or reporting, while remaining available to regulators if required.
- **Implementation note:** Column-level encryption for Emirates ID and DOB is on the Day 60 plan — this would make the pseudonymisation step cryptographically clean rather than relying on application-layer masking.

4 Sharia Compliance Considerations

How the data model currently handles this

Islamic finance prohibits riba (interest) and structures products around profit-sharing arrangements — Murabaha (cost-plus sale), Ijara (lease), Musharaka (equity partnership). The Part 1 data model has a *product_type* field on *gold.credit_decision_inputs* which currently supports three values: personal_finance, bnpl, credit_card_alternative. Extending this to distinguish Sharia-compliant variants is straightforward — but the more significant work is in the decision inputs themselves.

What the data model needs to change

Conventional credit signals don't translate directly to Sharia-compliant products. Specifically, the following fields need to be added or reinterpreted:

Data Point	Why It's Needed for Sharia Products
profit_rate (not interest_rate)	Murabaha charges a profit margin, not interest. Storing as interest_rate misrepresents the product and fails Sharia
asset_type & asset_value_aed	Ijara and Murabaha are asset-backed. The asset must exist and be Sharia-permissible (no alcohol, weapons, pork
financing_structure	Enum: murabaha | ijara | musharaka | tawarruq. Each has different risk and repayment profiles.
halal_income_declaration	Customer declares income source is Sharia-compliant. Relevant for high-value financing where income source ma
shariah_board_approval_ref	Each product structure must be approved by a Shariah board. The reference ID ties the decision to the approved p
customer_banking_type	Islamic banking vs conventional customer — affects which products can be offered and which credit bureau signals

Impact on existing credit signals

Some AECB signals are less predictive for Sharia customers. Credit card utilisation rate (*aecb_utilisation_rate*) is near-zero for customers who don't hold conventional credit cards. This field should not penalise Sharia-compliant customers in the scoring model. The data model supports this cleanly since it's stored separately per decision — the scoring model layer can weight it differently based on *financing_structure*.

5 What I Cut & Why — 48-Hour Scope Decisions

What I deliberately left out

What was cut	Why it was cut	When I'd add it
Kafka / real-time streaming	AECB is batch anyway and fraud API is synchronous per-re[illegible]	At 100K+/day [illegible]
Debezium CDC (full change capture)	Replaced with simpler updated_at watermark approach. De[illegible]	Day 60–90 [illegible] careful offset
Column-level PII encryption (Emirates ID, DOB)	Schema is designed to accommodate it (dedicated columns, no JSONB mixing) [illegible]	Day 60 [illegible] adds key management com
ML feature store / model training pipeline	The rule-based credit policy runs fine on raw silver fields. Bu[illegible]	Day 90 [illegible] days of producti
Multi-region disaster recovery	Single region (me-south-1) with Multi-AZ RDS and S3 cross-AZ replication covers the failure modes realistic for launch.	Before scaling beyond UAE
Redshift / data warehouse	Athena querying S3 Parquet is sufficient for analytics at curr[illegible]	Day 90 if Athena p95 query latency exceeds 10s [illegible] Day 90 if

My first five post-launch improvements, in priority order

- **1. GIN index on silver.customer_master.full_name** — pg_trgm similarity() does a sequential scan right now. A GIN trigram index cuts name matching from O(n) to near-constant time. This is the single highest-effort-to-reward fix.
- **2. Column-level encryption for Emirates ID and DOB** — currently stored in plaintext. Using pgcrypto or application-layer AES-256 before the Day 60 compliance checkpoint.
- **3. Velocity fraud alerting** — if the same customer_key appears in more than 3 applications within 7 days, that's a fraud ring signal. Easy to add as a gold-layer view; currently not in the DQ scorecard.
- **4. Sharia product_type extension** — add murabaha, ijara, musharaka to the product_type enum and add the six Sharia-specific fields to the gold snapshot table.
- **5. Automated CBUAE monthly report** — the data is all there in gold; it just needs a scheduled Athena query and a PDF export. Currently this would be manual.

> *The 48-hour constraint forced good discipline: build the immutable audit trail and the identity resolution correctly from day one, because retrofitting those is painful. Everything else — encryption, streaming, ML — can be layered on top of a clean foundation.*